

Code Assessment of the strkBTC Bridge Smart Contracts

April 29, 2026

Produced for



by



Contents

1	Executive Summary	3
2	Assessment Overview	5
3	Limitations and use of report	10
4	Terminology	11
5	Open Findings	12
6	Resolved Findings	13
7	Informational	15
8	Notes	16

1 Executive Summary

Dear all,

Thank you for trusting us to help Starkware with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of strkBTC Bridge according to [Scope](#) to support you in forming an opinion on their security risks.

Starkware implements a Bitcoin-to-Starknet bridge that enables users to mint wrapped Bitcoin (strkBTC) tokens on Starknet by depositing BTC, and redeem strkBTC for BTC through a quorum-based attestation mechanism managed by trusted signers.

The most critical subjects covered in our audit are functional correctness, access control, and security of assets. Functional correctness is satisfactory. Access control is satisfactory with proper role separation between governance, signers, and users. Security of assets relies on the trusted signer model.

The general subjects covered are event handling and documentation. Both subjects are satisfactory.

In summary, we find that the codebase provides a good level of security, assuming correct behavior of all trusted roles. The system's security heavily depends on the trustworthiness of governance and the signer quorum.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	1
Code Corrected	1



2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the strkBTC Bridge repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	30 March 2026	134b36ea35bce5d72f365ef03d228d868632cf33	Initial Version
2	6 April 2026	8829f3d2c93e91a51282bd297c0ae3d31b51e353	Intermediate Version
3	7 April 2026	482f932ca49fdbc3624d8d36834018c8aae802cb	Final Version

For the Cairo contracts, compiler version 2.15.1 was used. At the time of this review (April 2026), Starknet v0.14.1 was live on mainnet. This review cannot account for future changes and possible bugs in Starknet and its libraries.

The following files were in scope:

```
./contracts/bridge/src/bridge.cairo
./contracts/bridge/src/errors.cairo
./contracts/bridge/src/events.cairo
./contracts/bridge/src/interface.cairo
./contracts/bridge/src/lib.cairo
./contracts/bridge/src/utils.cairo

./contracts/registry/src/errors.cairo
./contracts/registry/src/events.cairo
./contracts/registry/src/interface.cairo
./contracts/registry/src/lib.cairo
./contracts/registry/src/registry.cairo
./contracts/registry/src/utils.cairo

./contracts/token/src/lib.cairo
./contracts/token/src/token.cairo
```

2.1.1 Excluded from scope

Any file not listed above is excluded from the scope. In particular, the starkware_utils and openzeppelin libraries are out of scope and assumed to work as documented. The Bitcoin side of the bridge is also out of scope, including the multisig scripts, UTXO management, and the off-chain signer nodes that witness deposits and broadcast withdrawal transactions.

2.1.2 Assumptions

This assessment assumes the validity of the assumptions documented by the client in the strkBTC Bridge specifications. Issues arising from violations of these assumptions are outside the scope of this audit. Key assumptions include:

- Signers are fully trusted to act honestly and correctly at all times.
- Bitcoin data provided by signers (transaction IDs, public keys, addresses) is valid and correctly formatted.
- Governance maintains proper configuration of signers, users, and quorum.
- Post-deployment role configuration and bridge initialization is completed correctly.

2.2 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the [Assessment Overview](#).

Starkware offers strkBTC Bridge, a Bitcoin-to-Starknet bridge enabling users to mint wrapped Bitcoin tokens (strkBTC) on Starknet by depositing BTC, and to redeem strkBTC for BTC by initiating withdrawals. Deposits are confirmed by a quorum of trusted off-chain signer nodes that witness Bitcoin transactions. Withdrawals are initiated on-chain by authorized users (LPs) and completed off-chain by signers co-signing a Bitcoin transaction. Governance roles control signer registration, user authorization, quorum, and contract upgrades.

2.2.1 Bridge Contract

The Bridge contract is the coordination hub of the system. It manages deposit witnessing, withdrawal initiation, signer and user registration, and quorum configuration. It interacts with both the token contract (to mint and burn strkBTC) and the registry contract (to propagate signer changes). The bridge holds the APP_GOVERNOR role in the registry, making it the sole authority over the registry's signer set.

Deposit witnessing follows a quorum-based attestation model. Each deposit is uniquely identified by a `deposit_id` derived as the Poseidon hash of (`btc_txid`, `vout`, `amount`, `destination_address`). Registered signers call `witness_deposit()` to attest that a Bitcoin deposit occurred. The contract records each signer's attestation in an iterable map per deposit. After each new witness, if the number of non-blacklisted witnesses meets or exceeds quorum (minimum 2) and the deposit has not yet been minted, it calls `permissioned_mint()` on the token contract and marks the deposit as minted. Writing the same signer's witness twice is idempotent (unless quorum has changed). A revoked signer's witnesses are excluded from the count retroactively.

Withdrawals are initiated by authorized users. `request_withdraw()` verifies the caller is an authorized user, and the amount meets the configurable minimum threshold. It then calls `permissioned_burn()` on the token contract to destroy the caller's strkBTC, and emits `WithdrawRequested`. Each off-chain signer independently observes this event, builds the same Bitcoin transaction deterministically, signs it, and posts their signatures to the Registry contract.

Signer management is centralized in the bridge. The functions `register_signer`, `remove_signer`, and `revoke_signer` update the bridge's own signer storage and propagate the corresponding call to the registry contract.

The bridge uses a two-step initialization. The constructor sets up roles and replaceability, while `init_bridge` (callable once, by the APP_GOVERNOR) configures the token address, registry address, quorum, and minimum withdrawal amount.

The contract provides the following functions:

User-facing:



- `request_withdraw(amount, btc_destination)`: Only authorized users. Burns amount strkBTC from the caller and emits `WithdrawRequested`.

Signer-facing:

- `witness_deposit(btc_txid, vout, amount, destination_address)`: Only registered signers. Records the caller's witness for the deposit. Emits `DepositWitnessed` unconditionally. If the non-blacklisted witness count reaches quorum and the deposit has not been minted, mints strkBTC to `destination_address` and emits `DepositMinted`.
- `is_witnessed(btc_txid, vout, amount, destination_address, signer)`: Callable by anyone. Returns whether signer has witnessed the specified deposit.

Governance. Following functions can only be called by APP_GOVERNOR.

- `init_bridge(token_address, registry_address, quorum, min_withdraw_amount)`: One-time initialization. Enforces that quorum is set to at least 2.
- `register_signer(signer, btc_public_key)`: Registers a signer in Bridge and Registry. Reverts if the signer is blacklisted, the BTC key is already in use by another signer, or the key is empty.
- `remove_signer(signer)`: Deregisters a signer from both contracts without blacklisting. Prior witnesses remain valid.
- `revoke_signer(signer)`: Deregisters a signer and blacklists their Starknet address (on the Bridge) and BTC public key hash (on the Registry).
- `register_user(user)`: Authorizes a user to request withdrawals.
- `remove_user(user)`: Revokes a user's withdrawal authorization.
- `set_min_withdraw_amount(min_withdraw_amount)`: Updates the minimum withdrawal amount. Can be set to zero to disable the minimum check.
- `set_quorum(quorum)`: Updates the quorum. Must be at least 2.
- `is_signer(signer)`, `is_user(user)`, `get_min_withdraw_amount()`, `get_quorum()`: View functions.

2.2.2 Registry Contract

The Registry contract stores Bitcoin signatures for pending withdrawals and maintains the authorized signers' BTC public key mappings. Its primary consumers are the off-chain signer nodes, which call `sign_withdraw()` after building the Bitcoin PSBT for a pending withdrawal. The registry does not validate signature content or enforce any binding between a submitted `raw_tx` and a `WithdrawRequested` event emitted by the bridge.

Each withdrawal is identified by a `withdraw_id`, the Poseidon hash of the raw Bitcoin transaction bytes. For each withdrawal, the contract maintains an ordered list of BTC public keys that have signed (`withdraw_id_to_signers`) and a separate per-signer list of signatures keyed by (`withdraw_id, btc_pubkey_hash`). When a signer calls `sign_withdraw`, the contract appends their public key to the signers list on first submission, or replaces their previous signatures on re-submission. It then aggregates all stored signatures for the withdrawal, skipping any whose BTC public key hash appears in the blacklist, and emits `WithdrawSigned` with the full aggregated set. Off-chain relayers consume this event and attempt to broadcast the Bitcoin transaction once sufficient signatures are present.

Signer management in the registry is controlled exclusively by the bridge contract. `register_signer()` checks that the BTC public key hash is not blacklisted before writing. `revoke_signer()` blacklists the BTC public key hash and removes the signer, ensuring that previously submitted signatures from a revoked signer are excluded from all future `WithdrawSigned` aggregations.

The contract provides the following functions:

- `sign_withdraw(raw_tx, signatures)`: Only registered signers. Records or replaces the caller's signatures for the withdrawal identified by the Poseidon hash of `raw_tx`. Emits `WithdrawSigned` with all non-blacklisted signer signatures aggregated. Reverts if `raw_tx` or `signatures` is empty.
- `has_signed_withdraw(raw_tx, btc_pubkey)`: View function returning whether the owner of `btc_pubkey` has submitted at least one signature for the specified withdrawal.
- `register_signer(signer, btc_pubkey)`: Only `APP_GOVERNOR` (bridge contract). Associates signer with `btc_pubkey`. Reverts if the BTC public key hash is blacklisted.
- `remove_signer(signer)`: Only `APP_GOVERNOR`. Clears the signer's BTC public key entry without blacklisting.
- `revoke_signer(signer)`: Only `APP_GOVERNOR`. Blacklists the signer's BTC public key hash and removes the signer.
- `is_signer(signer)`: View function returning whether `signer` has a non-empty BTC public key registered.

2.2.3 Token (strkBTC)

The Token contract implements the strkBTC ERC-20 token representing wrapped Bitcoin on Starknet. It uses 8 decimals to match Bitcoin's native precision. Minting and burning are restricted to a single permitted_minter address stored at construction. In the expected deployment this is the bridge contract. The permitted_minter is a storage variable, not a role. It is fixed at deployment and cannot be changed without a contract upgrade.

The contract extends OpenZeppelin's `ERC20Component`, `AccessControlComponent`, and `SRC5Component`, along with StarkWare's `RolesComponent` for governance role management and `ReplaceabilityComponent` for time-locked upgrades.

The contract provides the following functions:

- `permissioned_mint(account, amount)`: Only permitted_minter. Mints amount tokens to account. Called by the bridge when a deposit reaches quorum.
- `permissioned_burn(account, amount)`: Only permitted_minter. Burns amount tokens from account. Called by the bridge when a user requests a withdrawal.
- `is_permitted_minter(account)`: View function returning whether account equals the permitted_minter address.
- `increase_allowance(spender, added_value)` / `decrease_allowance(spender, subtracted_value)`: Allowance helpers that are callable by anyone.
- Standard ERC-20 functions (`transfer`, `transferFrom`, `approve`, `allowance`, `balanceOf`, `totalSupply`) via `ERC20Component`.

2.2.4 Trust Model

Governance Admin: Fully trusted. Controls role assignment across all three contracts. Can steal funds via: (1) granting `APP_GOVERNOR` to a malicious address to register fake signers and mint unbacked strkBTC, (2) performing malicious contract upgrades via `ReplaceabilityComponent` on any of the three contracts.

App Governor: Fully trusted. Manages signers, users, quorum, and minimum withdrawal amount on the bridge. Can steal funds via: (1) registering colluding signers to witness non-existent Bitcoin deposits, triggering unbacked minting, (2) lowering quorum to 2 (the enforced minimum) to reduce the collusion threshold, (3) revoking all legitimate signers to permanently block withdrawals and trap user funds.



Because the bridge holds APP_GOVERNOR in the registry, the entity controlling the bridge's APP_GOVERNOR role effectively controls the registry's entire signer set.

Signers: Partially trusted. Can witness deposits but cannot unilaterally mint tokens. A quorum of malicious signers could mint unbacked strkBTC by submitting fabricated deposit parameters to `witness_deposit()`. Removed (non-revoked) signers retain their past witness attestations, which continue to count toward quorum for any unminted deposits.

Authorized Users (LPs): Partially trusted. Can burn only their own strkBTC to initiate withdrawals. A malicious user cannot affect other users' funds. The whitelist approach prevents arbitrary addresses from creating withdrawal requests.

End Users (strkBTC Holders): Untrusted. Can transfer tokens freely using standard ERC-20 functions. Cannot mint, burn, or initiate withdrawals unless registered as an authorized user.

External Dependencies: The system relies on off-chain signer nodes to observe Bitcoin and Starknet state correctly and act honestly. There is no on-chain oracle or price feed. If all signers become unavailable, deposits cannot be minted and withdrawals cannot be broadcast, locking funds. Bitcoin transaction validity, confirmation depth (6 blocks per architecture docs), and LP authorization are assessed entirely off-chain by signers. The contracts do not verify these properties.

Upgradeability: All three contracts use StarkWare's `ReplaceabilityComponent`, implementing time-locked upgrades. The `upgrade_delay` is set at deployment and enforces a mandatory waiting period before a proposed implementation takes effect.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the [Resolved Findings](#) section. The findings are split into these different categories:

- **Design**: Architectural shortcomings and design inefficiencies

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	0

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the [Open Findings](#) section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	1
• No Bounds on Signature Size or Count Code Corrected	
Informational Findings	3
• Documentation and Naming Inconsistencies Specification Changed • Redundant Event Emissions Code Corrected • Scope of Contract-Enforced Correctness Specification Changed	

6.1 No Bounds on Signature Size or Count

Design **Low** **Version 1** **Code Corrected**

CS-STRKBTC-003

The `Registry.sign_withdraw()` function accepts signatures without limits on individual signature length or the number of signatures per submission. A malicious or compromised signer could submit oversized or excessive signatures, bloating storage and increasing gas costs for subsequent `aggregate_signatures()` calls.

Since `aggregate_signatures()` is called on every `sign_withdraw()` and iterates over all stored signatures, in theory enough data could cause aggregation to exceed gas limits, blocking other signers from completing the withdrawal. Note that the app governor can mitigate this by blacklisting the offending signer, as blacklisted signers are skipped during aggregation.

Adding bounds would provide defense-in-depth.

Code corrected:

Starkware implemented an upper bound on both the number of signatures submitted and the individual signature length.

6.2 Documentation and Naming Inconsistencies

Informational **Version 1** **Specification Changed**

CS-STRKBTC-006

The architecture documentation, code comments, and state variable naming contain several inconsistencies:

- `is_deposit_witnessed()` is referenced in the architecture but the actual function is `is_witnessed()`.
 - `is_withdraw_signed()` is referenced in the architecture but the actual function is `has_signed_withdraw()`.
 - The quorum storage variable comment states it applies to "deposit and withdraw," but on-chain quorum enforcement only applies to deposits.
 - The Bridge contract names its signer-to-public-key mapping `signer_to_public_key`, while the Registry contract names the equivalent mapping `signers_to_pubkey`.
-

Specification changed:

Starkware resolved the documentation and naming inconsistencies.

6.3 Redundant Event Emissions

Informational **Version 1** **Code Corrected**

CS-STRKBTC-005

`register_user()` does not check whether the target address is already registered. Calling it on an existing user silently succeeds and emits a `UserRegistered` event, which misrepresents the state change to off-chain monitors.

Version 2:

In **Version 2**, `set_min_withdraw_amount()` and `set_quorum()` were updated to emit events. However, they do not check whether the new value differs from the current one, leading to redundant event emissions.

Code corrected:

Redundant event emissions have been mitigated.

6.4 Scope of Contract-Enforced Correctness

Informational **Version 1** **Specification Changed**

CS-STRKBTC-004

`Architecture.md` describes the signer model as follows:

Signer nodes – each signer independently watches both chains, witnesses deposits, signs withdrawals, and broadcasts fully-signed Bitcoin transactions. No signer coordinates with another – correctness is enforced by the contracts.

Specification changed:

Starkware removed "correctness is enforced by the contracts", which was inaccurate.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 Blacklisted Signer Signatures Require Off-Chain Filtering

Informational **Version 1** **Acknowledged**

CS-STRKBTC-001

`aggregate_signatures()` excludes signatures from blacklisted signers on-chain. However, if a signer is blacklisted after the `WithdrawSigned` event has been emitted, that event becomes stale. It still contains the now-blacklisted signer's signature. Off-chain components that construct the Bitcoin transaction should therefore check the current blacklist status and filter out signatures from blacklisted signers when finalizing the PSBT.

Acknowledged:

Starkware stated they will take this into consideration in the off-chain implementation.

7.2 Missing Events for Admin and Signature Management Functions

Informational **Version 1** **Code Partially Corrected**

CS-STRKBTC-002

Several admin functions and signature management functions do not emit events, making state changes unobservable on-chain by external monitors or indexers. The following functions are affected:

- `bridge.set_min_withdraw_amount()`: updates the minimum withdrawal amount without emitting an event
 - `bridge.set_quorum()`: updates the witness quorum threshold without emitting an event
 - `registry.write_signatures()`: silently drops existing signatures when overwriting entries for a withdraw ID
-

Code partially corrected:

Starkware updated `set_min_withdraw_amount()` and `set_quorum()` to emit events.

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Missing Validation for Bitcoin Data

Note Version 1

Several parameters accepting Bitcoin data lack length validation:

- `btc_txid` in `Bridge.witness_deposit()` accepts any length, though Bitcoin transaction IDs are exactly 32 bytes.
- `btc_pubkey` in `Registry.register_signer()` accepts any length, though Bitcoin public keys have fixed lengths depending on the format used.
- `btc_destination` in `Bridge.request_withdraw()` only checks for non-empty, but does not validate Bitcoin address format or length. If the address is malformed, signers cannot construct a valid Bitcoin transaction.

Starkware confirmed this is intentional as signers are trusted to provide correct data. Generally, adding format validation would be favorable as defense-in-depth.